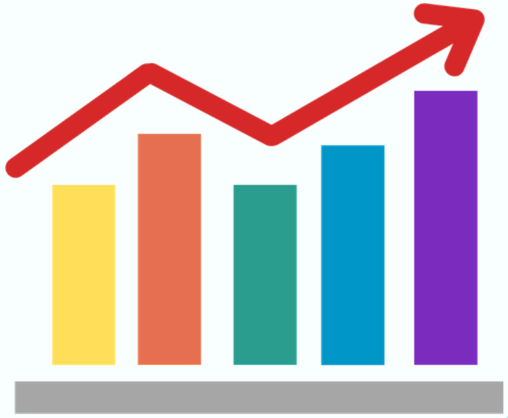
ADULT INCOME CENSUS DATASET

MACHINE LEARNING MODEL

Guna Thangavel
Gowri Ramalingam
Krishna Nanavaty



Problem Statement

Predicting Income Levels from Demographic & Work Data

Project Goal:

Develop a machine learning model to predict whether an individual earns more than $50K per year using demographic, education, occupation, and employment-related features, framed as a binary classification task (>50K vs ≤50K).

Project Significance:

Understanding income patterns helps in:

- Identifying key socioeconomic drivers of earning potential
- Supporting fair and data-driven workforce decisions
- Highlighting inequality across education, occupation, race, and work hours
- Enabling organizations and policymakers to make informed decisions

Decisions Enabled by the Model:

- HR & Recruitment: Better workforce planning and compensation fairness
- Policy & Social Research: Supports targeted policy interventions and social programs
- Business Analytics: Improves budgeting, workforce planning, and organizational strategy
- Education & Training: Helps design effective learning and development programs

ML Workflow (High-Level Overview):

Dataset Overview

Data Source Overview:

- Public Adult Census Income dataset from the UCI Machine Learning Repository (US 1994 census data).
- Each row represents one adult individual

Dataset Features & Variables:

- ~48K records with demographic and employment attributes.
- Examples of features: age, gender, education and education level, marital status, occupation, workclass, hours worked per week, capital gain/loss, native country.
- Target variable: income category (>50K vs ≤50K).

Key Data Characteristics:

- Mix of numeric (age, hours_per_week, capital_gain, etc.) and categorical variables (education, occupation, marital_status, etc.).
- Class distribution is imbalanced: more people earn ≤50K than >50K.
- Many features describe socio-economic status, making the dataset suitable for both prediction and fairness analysis

Hypothesis Testing

Purpose:To identify which demographic and socioeconomic factors significantly influence the probability of earning >50K.

Tests Used:

- Chi-Square Test → Categorical vs Categorical
- Two-Proportion Z-Test → Compare success rates between two groups
- Logistic Regression → Continuous predictors vs binary income outcome

Significant Factors Affecting Income (>50K):

- Education level
- Workclass
- Occupation × Hours worked
- Age
- Race
- Marital status
- Education × Hours interaction

Overall Conclusion:

Income is not random — it is strongly shaped by a combination of:

- Education,
- Work environment,
- Demographics, and
- Work intensity.

These factors show statistically significant associations with earning more than 50K.

Exploratory Data Analysis (EDA)

Data Exploration Steps:

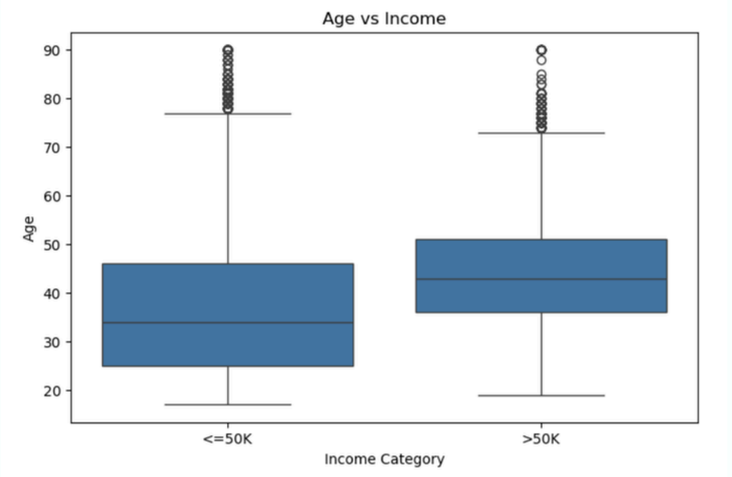
- Looked at summary statistics (mean, median, min, max) for key numeric features like age, hours per week, capital gain/loss.
- Checked class balance: how many people are in ≤50K vs >50K income.
- Reviewed data types and missing values to see what needed cleaning.

Insights:

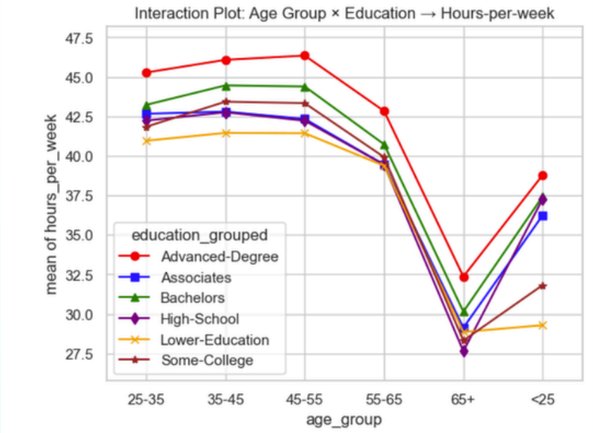
- Higher income is more common among middle-aged adults (not very young).
- People with higher education levels and more work hours show a larger share of >50K income.
- Non-zero capital gains are rare but strongly associated with >50K.

EDA – Key Visuals

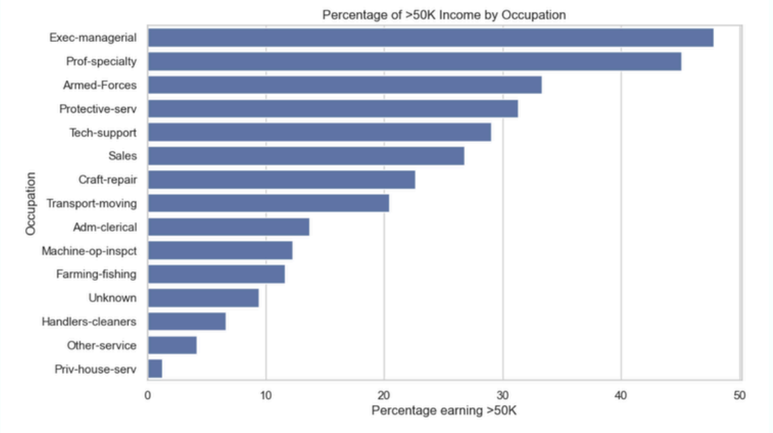
The dataset is imbalanced, with significantly more individuals earning ≤50K than >50K.



Higher income individuals tend to be older, with median age noticeably higher than the ≤50K group.



People with higher education consistently work more hours, especially in middle age groups.



High-income rates are highest in executive and professional roles, and lowest in service and manual labor jobs.

Data Cleaning

Consistency Checks Across Train & Test

- Ensured train and test datasets were combined before cleaning to maintain consistent preprocessing

Handling Missing Values

- Workclass → 2,799 missing → replaced with “Unknown”
- Occupation → 2,809 missing → replaced with “Unknown”
- Native Country → 856 missing → cleaned (“?” → NaN → “Unknown”)
- Other columns (Age, Education, Marital Status, Race, Sex, etc.) → No missing values

Outlier Analysis & Feature Engineering

- Capital Gain
  - 92% values = 0
  - Extreme value 99999 treated as capped
  - New feature: capital_gain_bin (0/1)
- Capital Loss
  - Right-skewed but valid
  - New feature: capital_loss_bin (0/1)
- Net Capital

New feature: net_capital = capital_gain – capital_loss

Data Cleaning(Cont..)

Standardization & Label Cleaning

- Income column
  - Cleaned inconsistent labels (>50K., <=50K.)
  - Standardized to ">50K" and "<=50K"
  - New target: income_binary (1 = >50K, 0 = <=50K)

- Native Country
  - Trimmed spaces, standardized categories

Columns With No Issues

- Age, Fnlwgt, Education, Education_num, Marital Status, Relationship, Race, Sex → No cleaning required

Duplicate Handling

- Identified 29 duplicate rows

Removed duplicates to avoid bias and data leakage

Feature Engineering

Feature Creator:

- Custom FeatureCreator adds interaction features (age × hours_per_week, education_num × hours_per_week) and age-group bins (Young, Mid, Senior, Elder).
- Enhances non-linear patterns and ensures consistent feature creation across train and test data.

Split Before Encoding & Scaling:

- Encoders/scalers learn statistics (categories, mean, std, PCA components).
- Fitting them on combined data leaks test-set information, causing invalid evaluation.
- Always split first, then fit only on training data.

Column Groups:

- Categorical → One-Hot Encoding
- Numeric → Median Imputation + Scaling
- Skewed Numeric → Log Transform + Scaling
- Ensures each feature receives the correct preprocessing treatment.

Preprocessing Pipeline:

- Built using a ColumnTransformer with three branches:
  - Numeric → median-impute + scale
  - Skewed numeric → log1p + scale
  - Categorical → most-frequent impute + OHE
- Produces a clean, standardized, fully numeric dataset ready for modeling.

Final Feature Names After Preprocessing:

- Final feature set includes:
  - Numeric features
  - Log-transformed skewed features
  - One-hot encoded categorical features

Expands into a model-ready feature matrix representing the exact inputs used during training and prediction.

Hyperparameter Tuning & Evaluation Metrics

Hyperparameter Tuning for feature Engineering

Model performance was improved using parameter tuning:

- Techniques: Grid Search / Random Search
- Key parameters tuned:
  - Number of trees (n_estimators)
  - Learning rate
  - Maximum depth

This optimization improved model accuracy and generalization.

Evalution Metrics:

To properly evaluate model performance on an imbalanced dataset, multiple metrics were used:

- Accuracy: Overall correctness of predictions
- Precision: How many predicted high-income individuals actually earn >50K
- Recall: Ability to correctly identify high-income individuals
- F1 Score: Balance between precision and recall
- ROC-AUC: Measures overall model discrimination ability

These metrics ensure a well-rounded evaluation beyond accuracy alone.

Models Used

We applied different models to understand the data better and find the best-performing approach

The Adult Census dataset contains both simple and complex patterns.

Numeric Models: Simple models that capture basic relationships in the data.

Tree-Based Models: Handles complex patterns by splitting data into smaller groups.

Logistic Regression

A simple model that predicts whether income is high or low based on the relationship between input features and the outcome.

Random Forest : Builds multiple decision trees and combines their predictions to give more reliable and accurate results.

XGBoost : A powerful model that improves predictions step by step by learning from previous mistakes.

LightGBM

A fast and efficient model that handles large data well and improves accuracy by learning from errors.

Support Vector Classifier (SVC)

This model separates data into different groups by finding the best way to divide them.

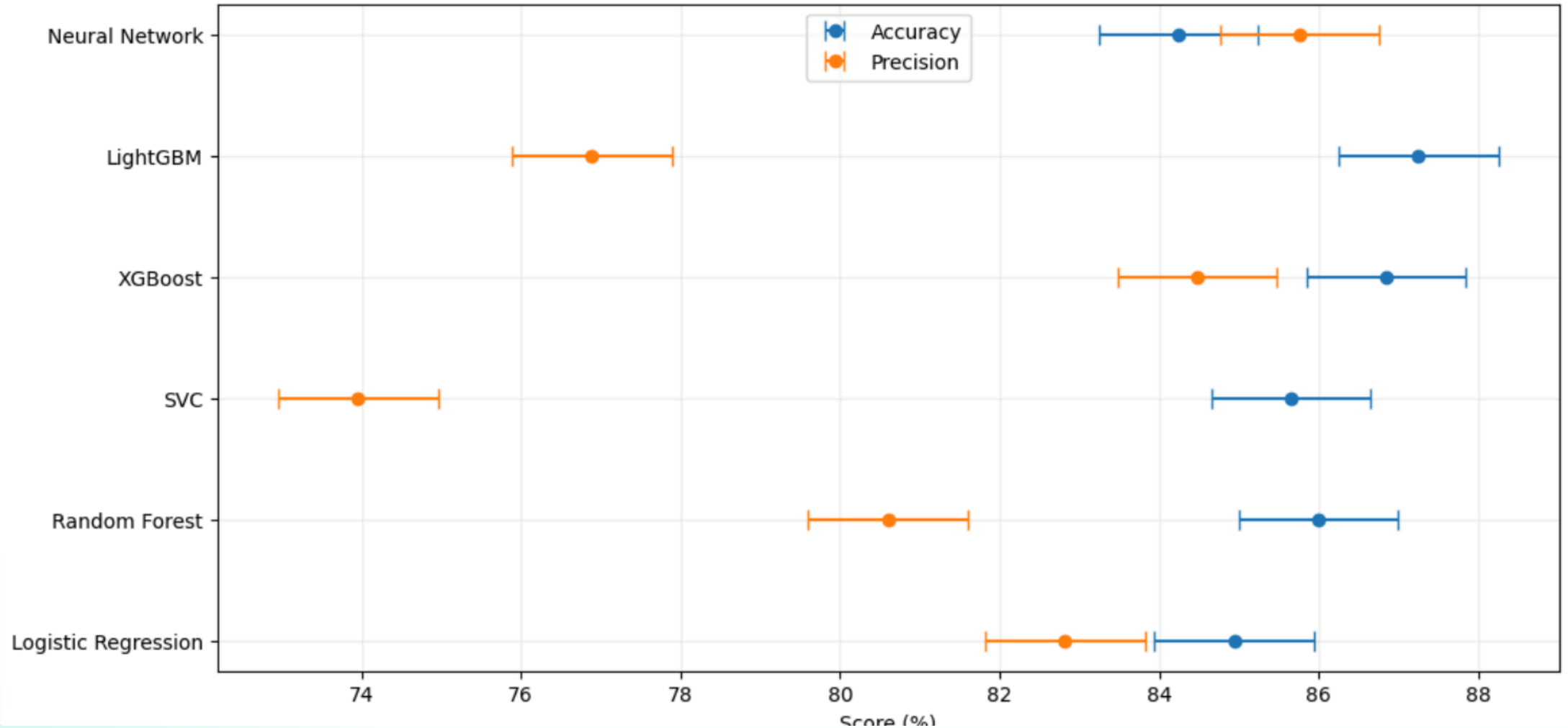
Separates the data into groups by finding the best way to distinguish between different classes.

Neural Network(Basic)

Learns patterns in the data through layers of connected nodes to make predictions.

Uses more layers or nodes to capture more complex patterns and improve prediction performance.

Model Performance Comparison



- The chart compares all models based on accuracy and precision.
- XGBoost and LightGBM show the highest accuracy among all models.
- XGBoost was selected as the final model due to its strong and well-balanced performance across all evaluation metrics.

Performance Dashboard

Model	Accuracy	Precision	Recall	F1-Score	Model Insights
Random Forest	0.86	0.806	0.5369	0.6445	Good balance but not the top performer
Logistic Regression (0.65)	0.8494	0.8282	0.4576	0.5895	Highly interpretable but lower recall
XGBoost (0.6)	**0.8685**	**0.8448**	0.5434	**0.6614**	**Best overall performance across metrics**
Neural Network	0.8424	0.8576	0.3993	0.5449	High precision but misses many >50K cases
SVC	0.8565	0.7396	**0.6063**	0.6663	Strong recall but lower precision and accuracy

Key Insights

Model Selection

The XGBoost model achieved the best overall results:

- High accuracy and precision
- Balanced recall
- Strong generalization on test data

It was selected as the final model for deployment.

- **XGBoost** delivered balanced performance across accuracy, precision, and recall, making it ideal for deployment.
- **SVC** excelled in recall, useful when capturing more positive cases is critical.
- **Neural Network** achieved high precision but predicted fewer positives, showing a conservative pattern.
- **Feature engineering** and proper categorical encoding (one-hot/target) improved model performance across all tree-based models.
- **Threshold tuning** allowed optimization, balancing precision and recall for business needs.
- **Tree-based models** consistently performed well, highlighting the strength of non-linear algorithms.

Logistic Regression served as a strong baseline, showing that simpler models still provide value.

Conclusion & Recommendation

1. Selected Model: XGBoost
    - Provides the best balance across accuracy, precision, and recall.
    - Reliable and consistent for real-world deployment.
2. Secondary Model: SVC (Optional)
    - Can be used when maximizing recall is a priority (capturing more positives).
3. Model Strategy:
    - Tree-based models (XGBoost, Random Forest, LightGBM) consistently outperform linear models on this dataset.
    - Threshold tuning can be applied to adapt model behavior to specific business goals.
4. Deployment Takeaway:
    - Use XGBoost as the primary model for production.
    - Monitor performance and adjust thresholds.